INF10025 Data Management and Analytics

# Task 4 – Pass Submission

***Student Name:*** Truong Ngoc Gia Hieu
***Student Number (Vietnam/Australia)***: SWS01217/105565520

---

## A. Task Pass4a

In the first task of the Pass section, following the ***provided example*** of the ***EMPLOYEE and BRANCH relational schema***, students are required to construct a corresponding ***relational schema*** based on the ***ERD*** between the ***CAMPAIGN and PLAYER entities***, ***clearly indicating all primary and foreign keys.***

**EMPLOYEE (EmpName, PhoneNo, BranchId)**
**Primary Key (EmpName)**
**Foreign Key (BranchId) references Branch**

**BRANCH (BranchId, BranchName)**
**Primary Key (BranchId)**

***Figure 1: Example of Employee and Branch Relational Schema.***

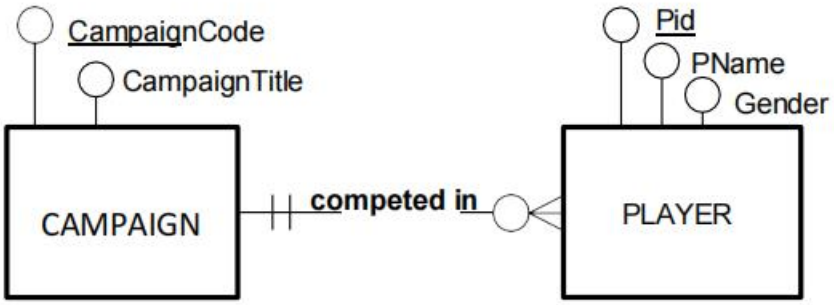



***Figure 2: Entity-Relationship Diagram for Campaign and Player Entities.***

CAMPAIGN (CampaignCode, CampaignTitle)
PRIMARY KEY (CampaignCode)

PLAYER (Pid, PName, Gender, CampaignCode)
PRIMARY KEY (Pid)
FOREIGN KEY (CampaignCode) references CAMPAIGN

***Figure 3: Relational Schema for Campaign and Player Entities.***

## B. Task Pass4b

Before creating the two following tables named ***Campaign5520*** and ***Player5520***, I have to check what tables have already been created in ***iSQL Junior*** by using the command '***show tables;'.***



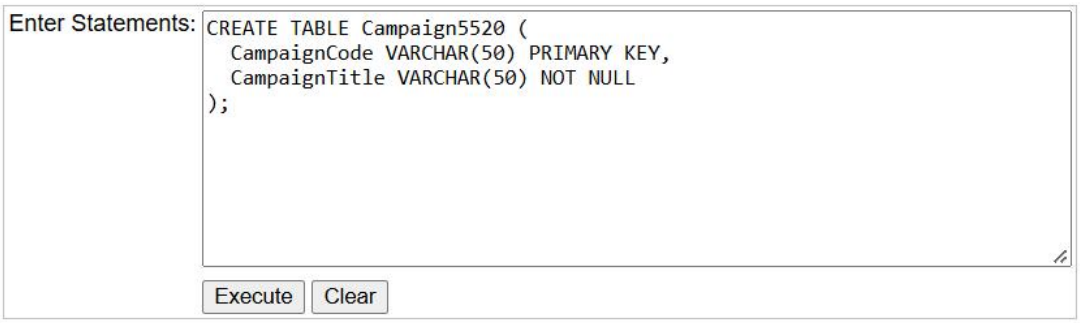

**SELECT**
**TABLE_NAME**
**FROM**
**ALL_TABLES**
**WHERE OWNER =**
**(SELECT USER**
**FROM DUAL)**

| TABLE_NAME |
|---|
| RATING5520 |
| MOVIE5520 |
| COLOURTYPE5520 |
| CASTING5520 |
| CAR5520 |
| ACTOR5520 |

OK. 6 rows selected.

***Figure 4: Checking Existing Tables in iSQL Junior.***

Then, I created the ***Campaign5520*** and ***Player5520*** tables respectively.




OK. 0 rows

***Figure 5: SQL Statement and Execution Result for Creating Campaign5520 Table.***

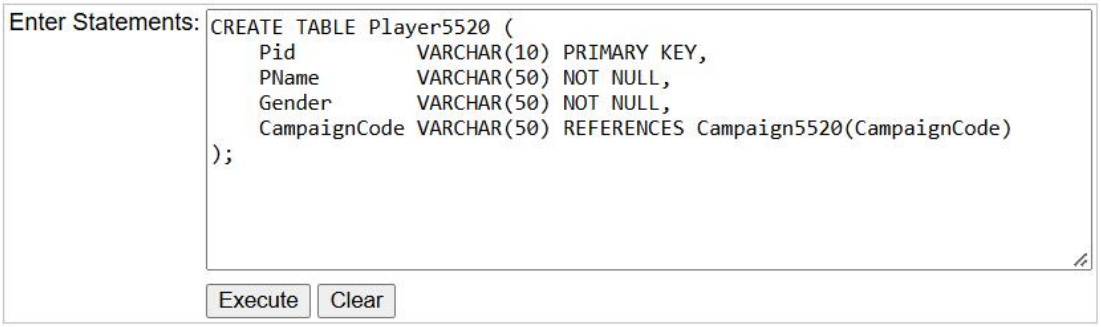



***Figure 6: SQL Statement and Execution Result for Creating Player5520 Table.***

## C. Task Pass4c

In the third task, I have to insert values into the ***Campaign5520*** and ***Player5520*** tables based on the provided data. However, I need to check again to ensure that both tables have already been created in iSQL Junior by using command '***show tables***;'.

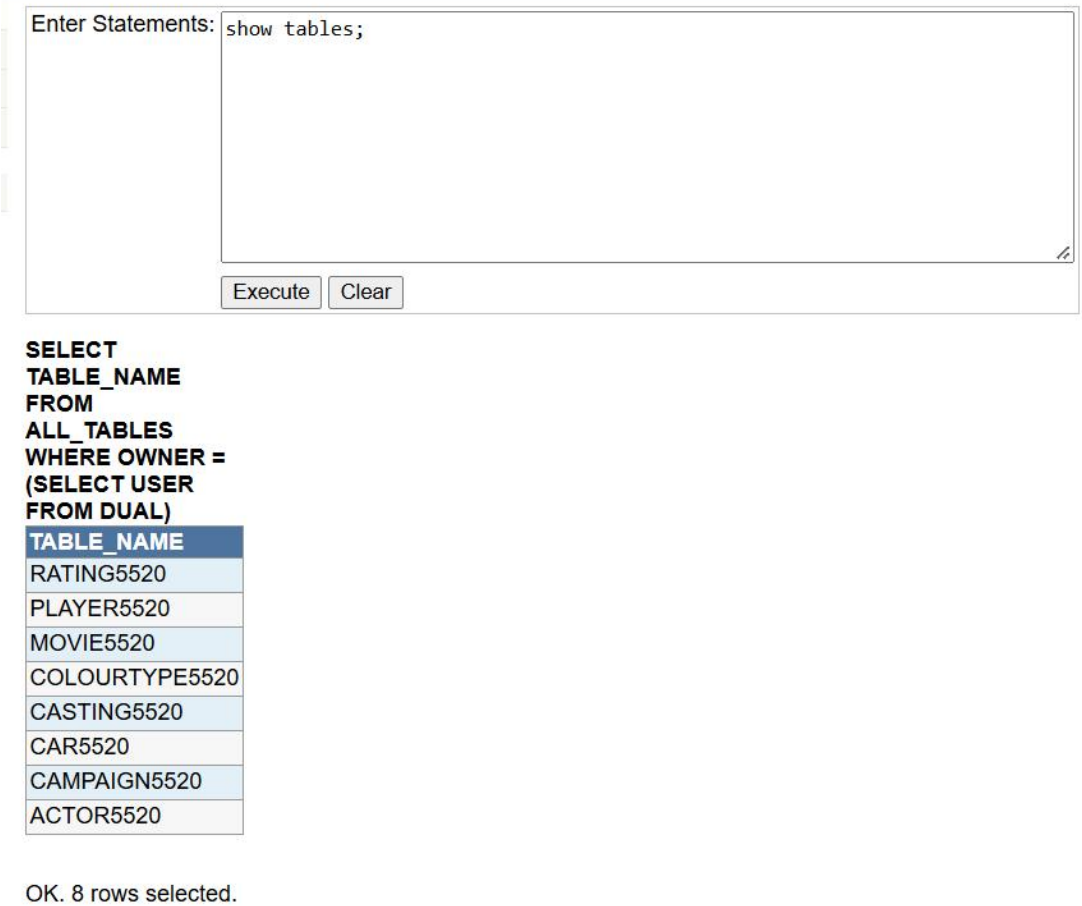



***Figure 7: Table List Verification.***

Notably, the iSQL Junior website allows me to insert data row by row, so I can just load the records sequentially.

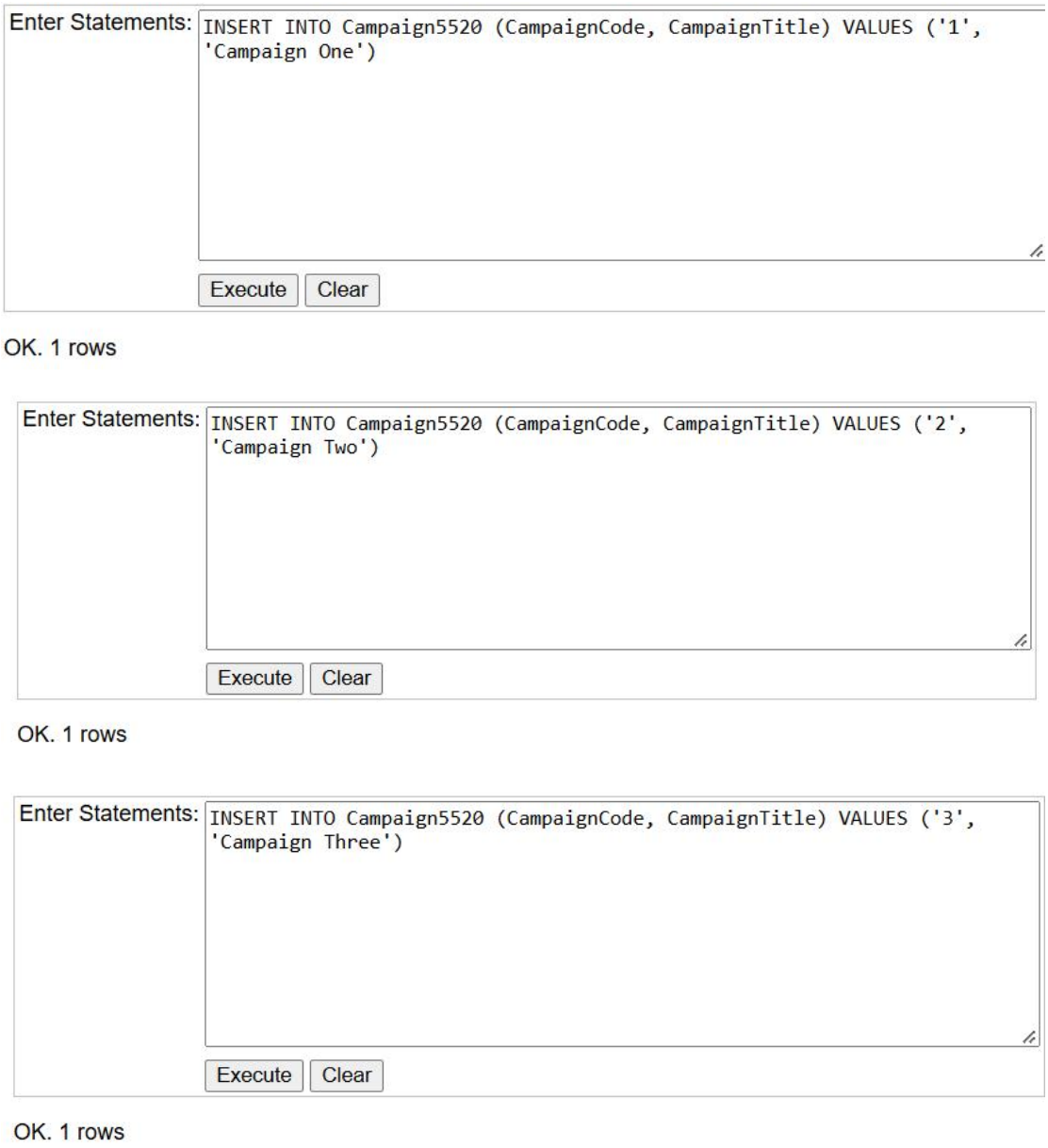
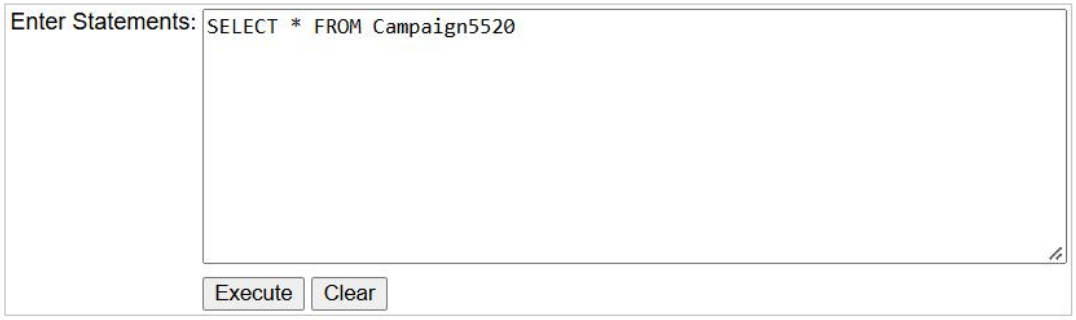



***Figure 8: Inserting Data into the Campaign5520 Table.***

After inserting all values into the table ***Campaign5520***, I used the command ***'SELECT * FROM Campaign5520'*** to **display all records in the table.**



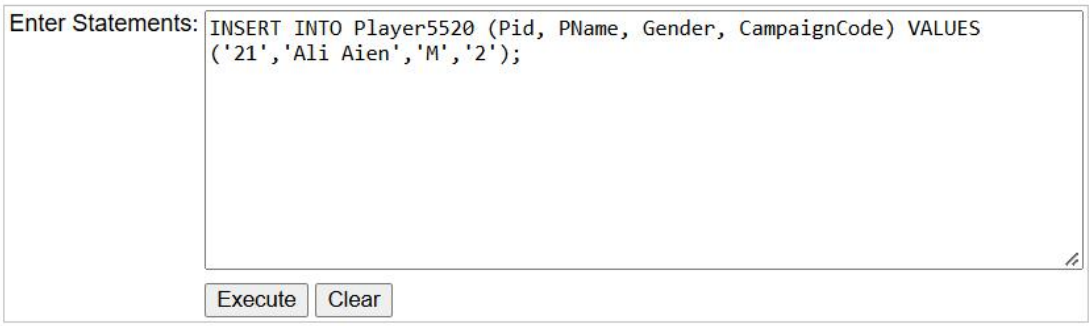

**SELECT * FROM Campaign5520**

| CAMPAIGNCODE | CAMPAIGNTITLE |
|---|---|
| 1 | Campaign One |
| 3 | Campaign Three |
| 2 | Campaign Two |

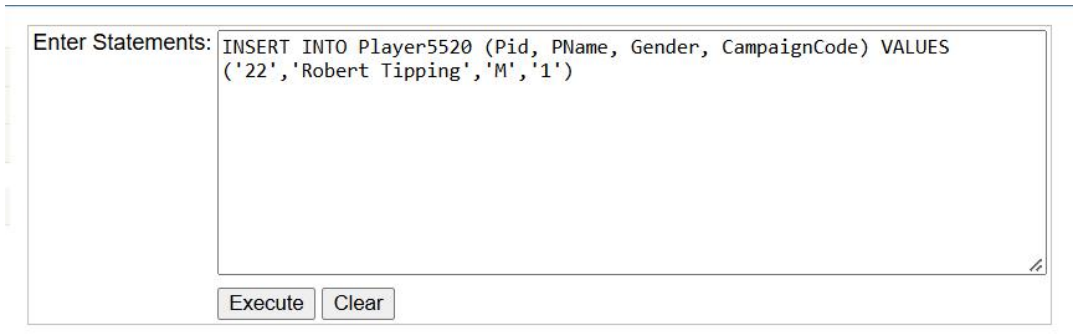
OK. 3 rows selected.

***Figure 9: Verification of Inserted Data in Campaign5520 Table.***

Next, I inserted sample data into the ***Player5520*** table based on the description.




OK. 1 rows




OK. 1 rows




***Figure 10: Inserting Data into the Player5520 table.***

Similarly, I used the command ***'SELECT * FROM Player5520'*** to **display all records in the table.**

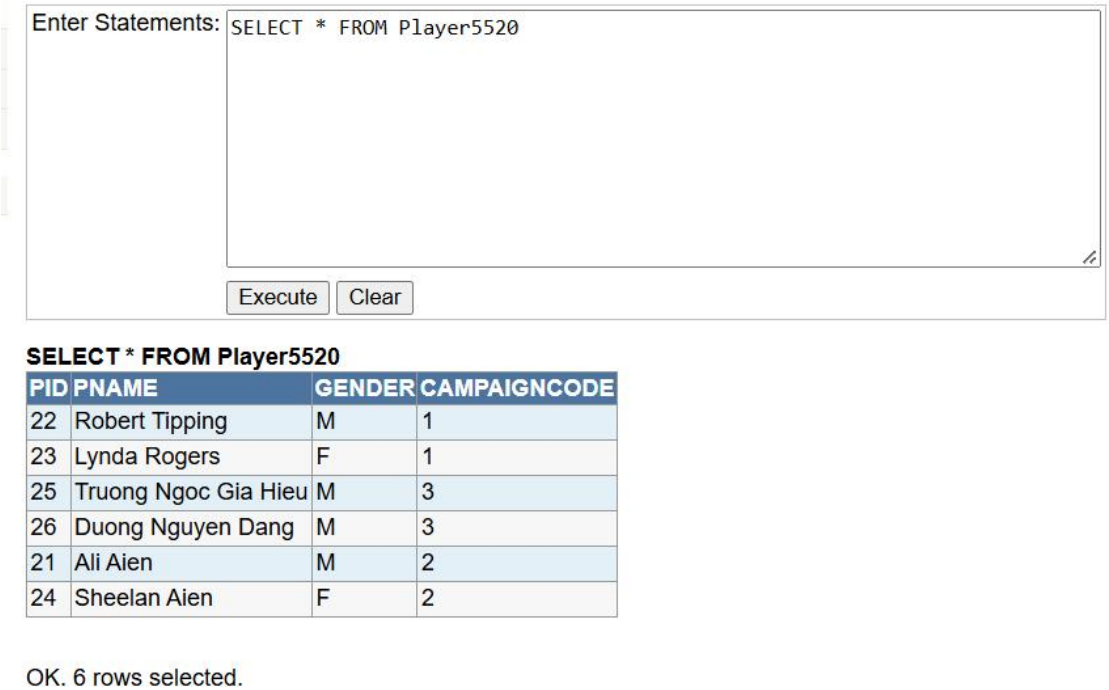



***Figure 11: Verification of Inserted Data in Player5520 Table.***

## D. Task Pass4d

Subsequently, in the next sub-task of the Pass submission, the task requires me to use **the** following commands for the ***Campaign5520*** and ***Player5520*** tables: ***'DESCRIBE Campaign5520', 'SELECT * FROM Campaign5520', 'DESCRIBE Player5520',*** and '***SELECT * FROM Player5520'.***

Enter Statements:
```
DESCRIBE Campaign5520;
SELECT * FROM Campaign5520
```
Execute Clear

**SELECT COLUMN_NAME, NULLABLE, DATA_TYPE FROM USER_TAB_COLS WHERE TABLE_NAME = 'CAMPAIGN5520'**

| COLUMN_NAME | NULLABLE | DATA_TYPE |
|---|---|---|
| CAMPAIGNCODE | N | VARCHAR2 |
| CAMPAIGNTITLE | N | VARCHAR2 |

OK. 2 rows selected.

**SELECT * FROM Campaign5520**

| CAMPAIGNCODE | CAMPAIGNTITLE |
|---|---|
| 1 | Campaign One |
| 3 | Campaign Three |
| 2 | Campaign Two |

OK. 3 rows selected.

Enter Statements:
```
DESCRIBE Player5520;
SELECT * FROM Player5520
```
Execute Clear

**SELECT COLUMN_NAME, NULLABLE, DATA_TYPE FROM USER_TAB_COLS WHERE TABLE_NAME = 'PLAYER5520'**

| COLUMN_NAME | NULLABLE | DATA_TYPE |
|---|---|---|
| PID | N | VARCHAR2 |
| PNAME | N | VARCHAR2 |
| GENDER | N | VARCHAR2 |
| CAMPAIGNCODE | Y | VARCHAR2 |

OK. 4 rows selected.

**SELECT * FROM Player5520**

| PID | PNAME | GENDER | CAMPAIGNCODE |
|---|---|---|---|
| 22 | Robert Tipping | M | 1 |
| 23 | Lynda Rogers | F | 1 |
| 25 | Truong Ngoc Gia Hieu | M | 3 |
| 26 | Duong Nguyen Dang | M | 3 |
| 21 | Ali Aien | M | 2 |
| 24 | Sheelan Aien | F | 2 |

OK. 6 rows selected.

***Figure 12: Structure and Contents of Campaign5520 and Player5520 Table.***

## E. Task Pass4e

In task Pass4e, first and foremost, I tried to insert a new record into the ***Campaign5520*** table by using the command ***'INSERT INTO Campaign5520 (CampaignCode, CampaignTitle) VALUES ('1', 'Campaign Four')'***.

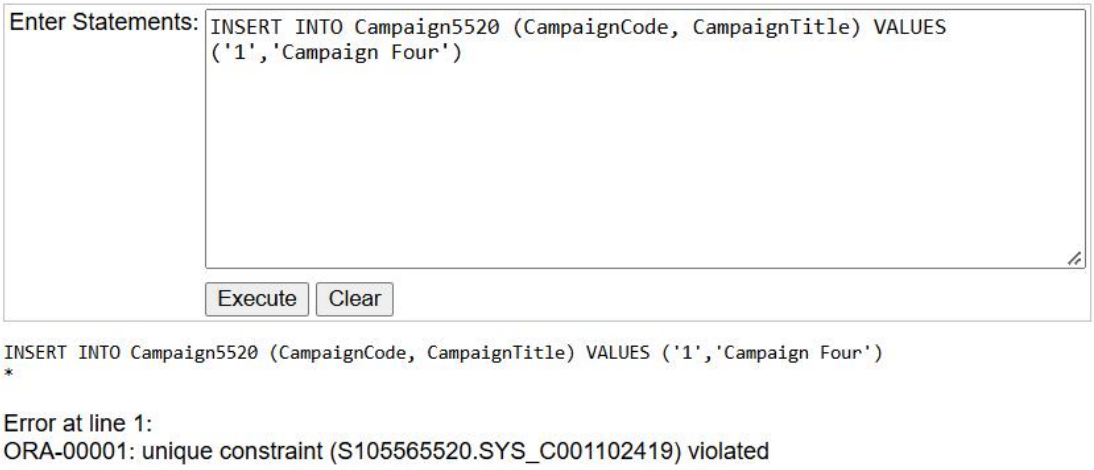



***Figure 13: Unique Constraint Violation Error on Campaign5520 Table.***

Based on the characteristics of a ***Primary Key*** which requires values to be unique, non-null, immutable, restricted to a single primary key per table, supported by automatic indexing, and able to serve as a foreign key reference, the statement resulted in an error when executed. Therefore, in the previous section, a record with ***CampaignCode = '1' ('1,','Campaign One') had already been inserted.***

Next, I tried to insert a new record into the ***Player5520*** table using the following command: ***'INSERT INTO Player5520 (Pid, PName, Gender, CampaignCode) VALUES ('27', 'Marcus Pontembelli', 'M', '4')***'.




***Figure 14: Foreign Key Integrity Constraint Violation Error on Player5520 Table.***

It can be seen from ***Figure 14*** that when executed, the website returns an error because the CampaignCode value '4' does not exist in the parent table (Campaign5520). According to the foreign key constraint defined in the schema, a child record cannot reference a primary key value that is missing from the parent table, thereby triggering the ORA-02291 integrity constraint violation (parent key not found).

Then, I tried to execute the command ***'DELETE FROM Campaign5520 WHERE CampaignCode = '2''***, as specified in the description.




***Figure 15: Foreign Key Constraint Violation Error when Deleting Parent Record.***

It is clear that the DELETE statement failed with an error because the record being removed (CampaignCode = '2') is still actively referenced by child records in the Player5520 table. In terms of maintaining referential integrity, Oracle prevents the deletion of a parent key as long as dependent child rows exist.

## F. Task Pass4f

According to the description of Task Pass4f, I have to write a single SQL statement that ***lists the player name and campaign title*** for ***every row in the Player5520 table***.




***Figure 16: SQL Statement for INNER JOIN Query on Player5520 and Campaign5520 Tables.***

**SELECT PName, CampaignTitle FROM Player5520 P INNER JOIN Campaign5520 C ON P.CampaignCode = C.CampaignCode**

| PNAME | CAMPAIGNTITLE |
|---|---|
| Robert Tipping | Campaign One |
| Lynda Rogers | Campaign One |
| Truong Ngoc Gia Hieu | Campaign Three |
| Duong Nguyen Dang | Campaign Three |
| Ali Aien | Campaign Two |
| Sheelan Aien | Campaign Two |

OK. 6 rows selected.

*Figure 17: Table Result Showing Joined Data from Player5520 and Campaign5520.*

## G. Task Pass4g

Next, I have to write a single SQL statement that counts the number of rows in the Player table.

Enter Statements:

```
SELECT COUNT(*) AS "Total Number of Rows"
FROM Player5520
```

Execute Clear

**SELECT COUNT(*) AS "Total Number of Rows" FROM Player5520**

| Total Number of Rows |
|---|
| 6 |

OK. 1 rows selected.

*Figure 18: SQL Statement and Execution Result for Counting Total Rows in Player5520 Table.*

Then, I have to write a single SQL statement that ***counts the number of rows grouped by CampaignCode in the Player5520 table.***

Enter Statements:

```
SELECT CampaignCode, COUNT(*) AS "Count per CampaignCode"
FROM Player5520
GROUP BY CampaignCode
```

Execute Clear

**SELECT CampaignCode, COUNT(*) AS "Count per CampaignCode" FROM Player5520 GROUP BY CampaignCode**

| CAMPAIGNCODE | Count per CampaignCode |
|---|---|
| 1 | 2 |
| 3 | 2 |
| 2 | 2 |

OK. 3 rows selected.

***Figure 19: SQL Statement and Execution Result for Grouping and Counting Records by CampaignCode.***

## H. Task Pass4h

Based on the description of Task ***Pass4h***, I opened the file named ***moviedatabaseDAD*** in ***Microsoft Access*** and then changed the table names, for instance, from ***MOVIE2833*** to ***MOVIE5520***, and similarly for the other tables.

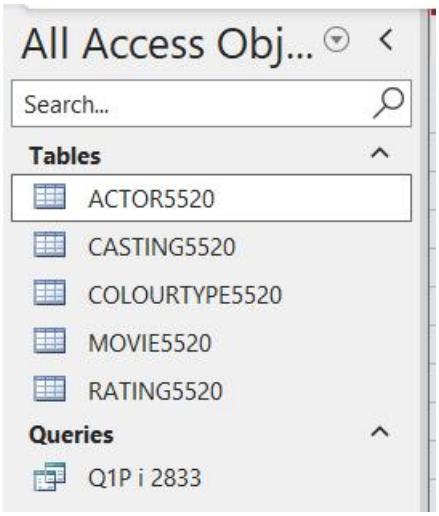



***Figure 20: Renamed Tables in Microsoft Access for Task Pass4h.***

Next, as specified in Task ***Pass4h***, I created a new ***Query Design*** named ***Task Pass4h*** in Microsoft Access, clicked on the SQL icon at the bottom right of the screen, and executed a single SQL statement (***'SELECT BirthCountry, COUNT(*) AS ActorCount FROM ACTOR5520 GROUP BY BirthCountry;'***) to count the number of rows grouped by ***BirthCountry in the ACTOR5520*** table. Finally, I derived a visual graph from the query results to effectively present and compare the distribution of actors across various countries, for instance, displaying higher concentrations in countries such as the USA compared to Germany.

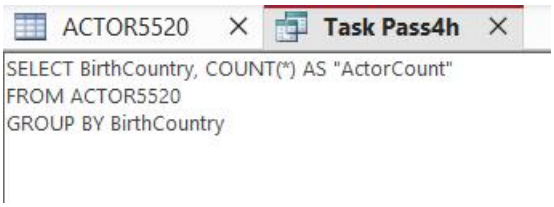
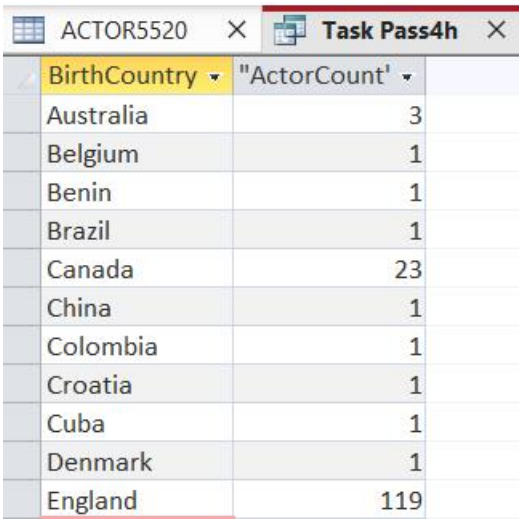



***Figure 21: SQL Statement in Microsoft Access to Count Actors by BirthCountry.***




| BirthCountry | "ActorCount' |
|---|---|
| Australia | 3 |
| Belgium | 1 |
| Benin | 1 |
| Brazil | 1 |
| Canada | 23 |
| China | 1 |
| Colombia | 1 |
| Croatia | 1 |
| Cuba | 1 |
| Denmark | 1 |
| England | 119 |

***Figure 22: Query Execution Result Displaying Actor Counts Grouped by BirthCountry.***

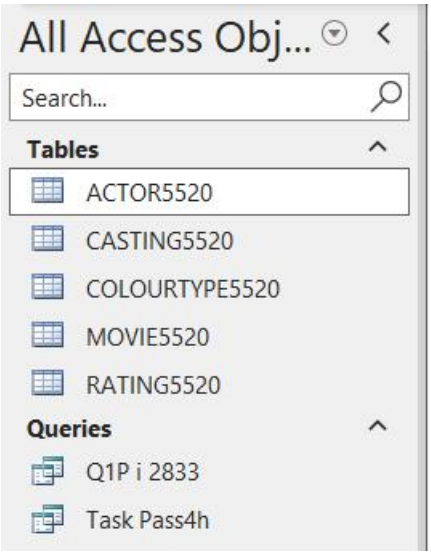



*Figure 23: Navigation Pane Displaying Saved Query 'Task Pass4h' in Microsoft Access.*

## I. Task Pass4i

In the task **Pass4i**, students are ***self-created*** to design a ***new query*** named ***Task Pass4i*** in Microsoft Access; specifically, by clicking on the SQL icon at the bottom right of the screen, I wrote a single SQL statement to ***retrieve specific columns (MovieTitle, Runtime, and RatingCode) from the MOVIE5520 table***. Therefore, the query filters movies for a limited range of runtimes (between 125 and 150 minutes) and a limited list of rating codes (for instance, 'G' and 'PG'), with the final displayed results systematically sorted in alphabetical order by movie title sequence.

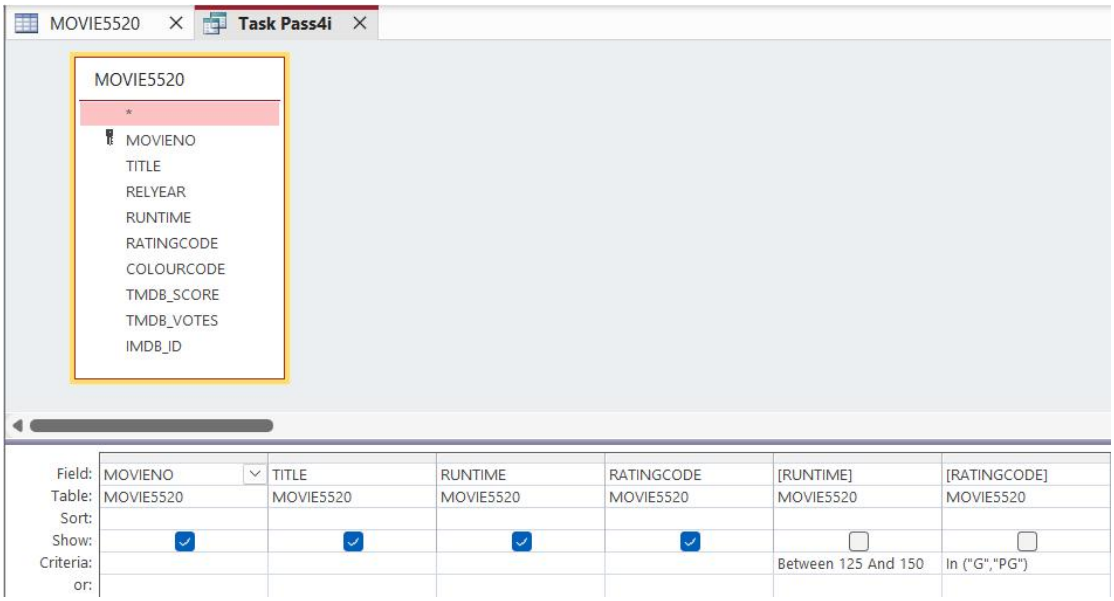



*Figure 24: Query Design View with Criteria Filters for Task Pass4i.*

Task Pass4i

| MOVIENO | TITLE | RUNTIME | RATINGCODE |
|---|---|---|---|
| 13 | Forrest Gump | 142 | PG |
| 22 | Pirates of the Caribbean: The Curse of the Black Pearl | 143 | PG |
| 299 | Ocean's Eleven | 127 | PG |
| 525 | Blues Borthers | 143 | PG |
| 558 | Spider-Man 2 | 127 | PG |
| 559 | Spider-Man 3 | 139 | PG |
| 568 | Apollo 13 | 140 | PG |
| 673 | Harry Potter and the Prisoner of Azkaban | 141 | PG |
| 675 | Harry Potter and the Order of the Phoenix | 138 | PG |
| 1633 | Fried Green Tomatoes | 130 | PG |
| 1726 | Iron Man | 126 | PG |
| 1858 | Transformers | 144 | PG |
| 2157 | Lost in Space | 130 | PG |
| 8818 | Evita | 134 | PG |
| 9277 | The Sting | 129 | PG |
| 10264 | Hamlet | 130 | PG |
| 11287 | A League of Their Own | 128 | PG |

*Figure 25: SQL View with Criteria Filters for Task Pass4i.*

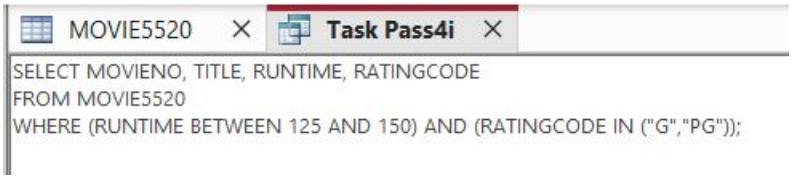



*Figure 26: Query Execution Result Displaying Filtered Movies for Task Pass4i.*

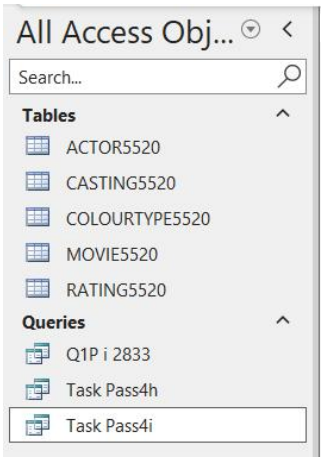



*Figure 27: Navigation Pane Displaying Saved Query 'Task Pass4i' in Microsoft Access.*

## J. Task Pass4j

In Task Pass4j, students are provided with a scenario describing the operations of the 'SwinEats' food delivery platform. Specifically, each delivery job requires a single driver and is defined by key attributes, such as a job ID, title, duration (in hours), and

description. Meanwhile, drivers are identified by their name, address, and mobile phone number.

Furthermore, the business domain permits a driver to perform multiple delivery jobs; conversely, the same job can also be undertaken by different drivers over time. Consequently, whenever a driver executes a job, specific transactional details must be recorded, including the execution date and a fixed fee. For instance, Jim completed Job #47 on April 15th, 2018, with a fee set at $10.

Based on this narrative, students are required to construct an Entity Relationship Diagram (ERD) adhering to standard unit notations, ensuring table names end with their 4-digit student ID (5520). Subsequently, the ERD must be mapped into a relational schema without surrogate keys, accompanied by two sample test data entries for each entity.

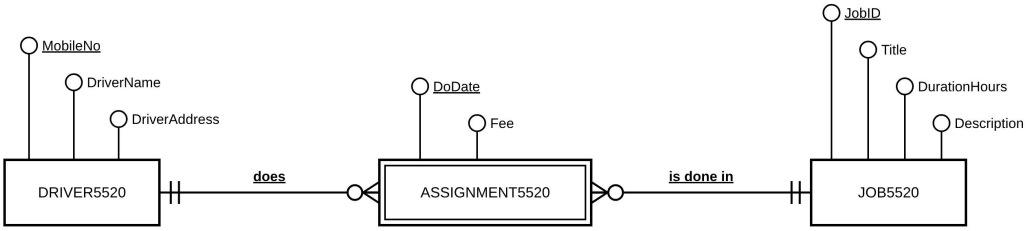



***Figure 28: Entity Relationship Diagram (ERD) for SwinEats Food Delivery System.***

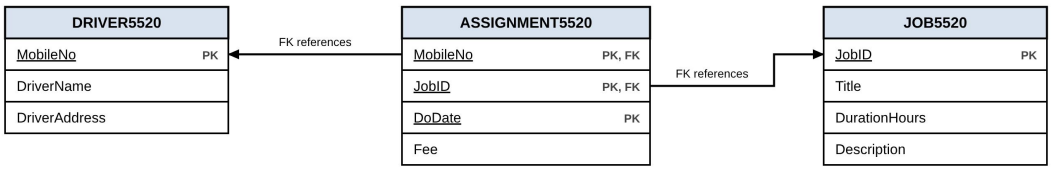



***Figure 29: Entity Relationship Model (ERM) for SwinEats Food Delivery System.***

DRIVER5520 (MobileNo, DriverName, DriverAddress)
Primary Key (MobileNo)

JOB5520 (JobID, Title, DurationHours, Description)
Primary Key (JobID)

ASSIGNMENT5520 (MobileNo, JobID, DoDate, Fee)
Primary Key (MobileNo, JobID, DoDate)
Foreign Key (MobileNo) references DRIVER5520
Foreign Key (JobID) references JOB5520

***Figure 30: Relational Schema for SwinEats Food Delivery System.***

From my perspective, the many-to-many relationship between drivers and jobs should be resolved by creating the weak entity ***ASSIGNMENT5520***, which links ***DRIVER5520*** and ***JOB5520*** through two one-to-many relationships. Furthermore, ***ASSIGNMENT5520*** borrows the identifiers ***MobileNo*** and ***JobID*** from its parent entities, and these become foreign keys in the relational schema. Consequently, its primary key is the composite (***MobileNo, JobID, DoDate***), because a single date alone cannot uniquely identify an assignment; for instance, different drivers can complete different jobs on the same day.

First and foremost, I created the above tables in the iSQL Junior website, then in Microsoft Access and .txt files in order to establish the relational database schema, test primary and foreign key constraints, and facilitate data migration.

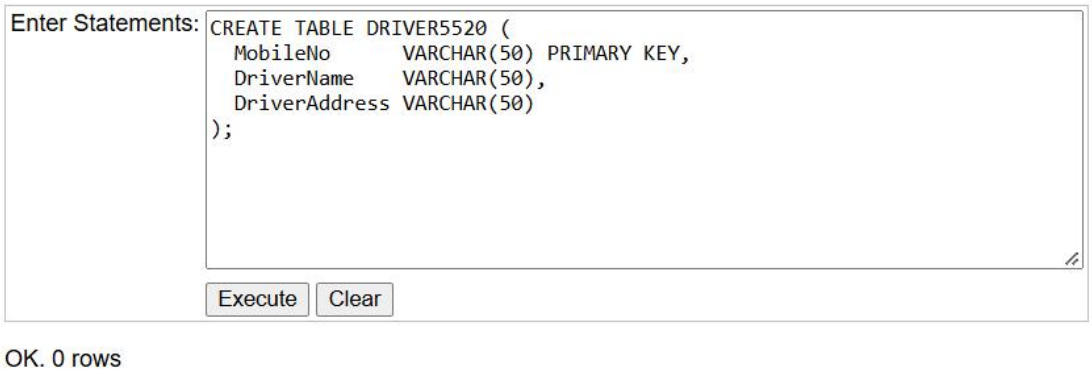



***Figure 31: Execution of DDL Statement for DRIVER5520 Table.***

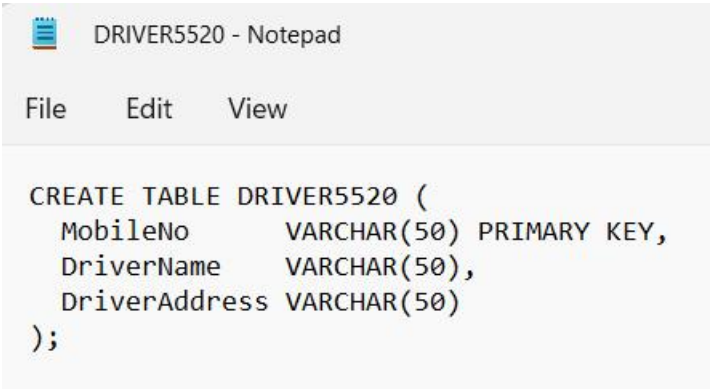



***Figure 32: SQL Script File for DRIVER5520 Table Creation.***

Enter Statements:
```
show tables;
```
Execute Clear

```
SELECT
TABLE_NAME
FROM
ALL_TABLES
WHERE OWNER =
(SELECT USER
FROM DUAL)
```

| TABLE_NAME |
|---|
| ACTOR5520 |
| CAMPAIGN5520 |
| CAR5520 |
| CASTING5520 |
| COLOURTYPE5520 |
| DRIVER5520 |
| MOVIE5520 |
| PLAYER5520 |
| RATING5520 |

OK. 9 rows selected.

*Figure 33: Verification of Existing Database Tables in iSQL Junior.*

Enter Statements:
```
INSERT INTO DRIVER5520 (MobileNo, DriverName, DriverAddress) VALUES
('0901234567', 'Jim Taylor', '12 Nguyen Hue St')
```
Execute Clear

OK. 1 rows

Enter Statements:
```
INSERT INTO DRIVER5520 (MobileNo, DriverName, DriverAddress) VALUES
('0912345678', 'Mai Nguyen', '45 Le Loi St')
```
Execute Clear

OK. 1 rows

*Figure 34: Inserting Test Data into DRIVER5520 Table.*




*Figure 35: Querying All Records from DRIVER5520 Table.*

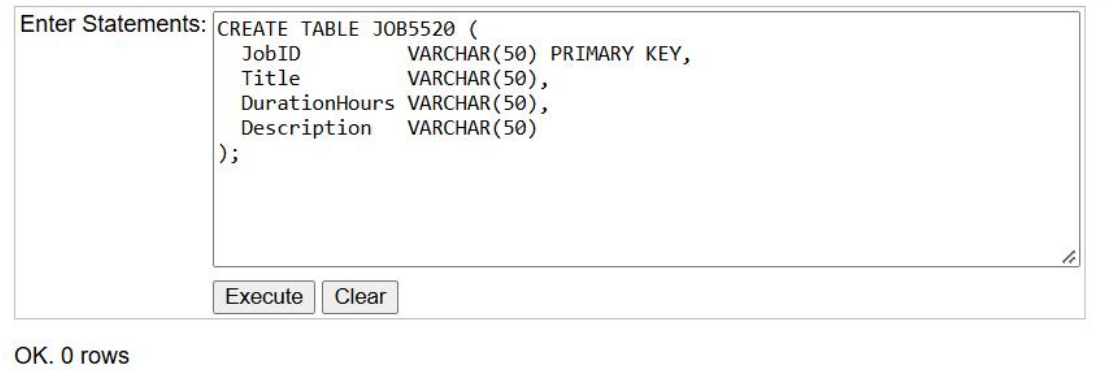



*Figure 36: Creation of JOB5520 Table in iSQL Junior.*

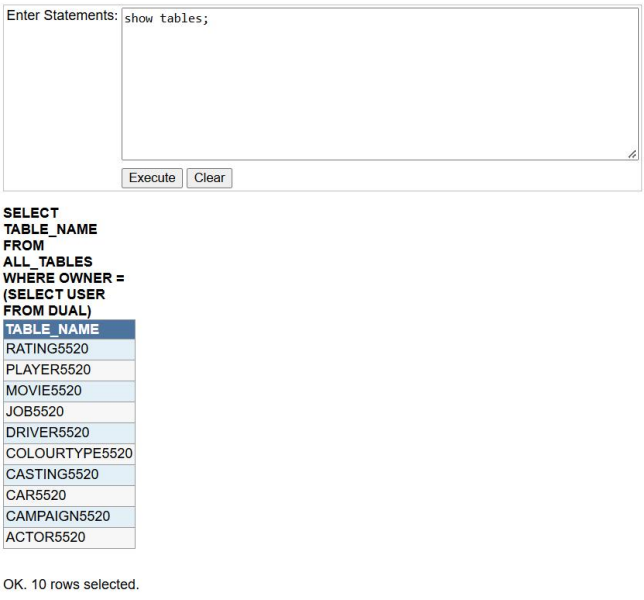



*Figure 37: Verification of Updated Table List Including JOB5520.*

Enter Statements:

```
INSERT INTO JOB5520 (JobID, Title, DurationHours, Description) VALUES
('52', 'Dinner Delivery', 3, 'Evening dinner deliveries')
```

Execute Clear

OK. 1 rows

*Figure 38: Executing Insert Statement for JOB5520 in iSQL Junior.*

Enter Statements:

```
SELECT * FROM JOB5520
```

Execute Clear

**SELECT * FROM JOB5520**

| JOBID | TITLE | DURATIONHOURS | DESCRIPTION |
|---|---|---|---|
| 47 | Lunch Delivery | 2 | Deliver office lunch orders |
| 52 | Dinner Delivery | 3 | Evening dinner deliveries |

OK. 2 rows selected.

*Figure 39: Querying All Records from JOB5520 Table.*

Enter Statements:

```
CREATE TABLE ASSIGNMENT5520 (
  MobileNo    VARCHAR(50),
  JobID       VARCHAR(10),
  DoDate      DATE,
  Fee         NUMBER(6,2),
  PRIMARY KEY (MobileNo, JobID, DoDate),
  FOREIGN KEY (MobileNo) REFERENCES DRIVER5520(MobileNo),
  FOREIGN KEY (JobID) REFERENCES JOB5520(JobID)
);
```

Execute Clear

OK. 0 rows

*Figure 40: Creating ASSIGNMENT5520 Table with Primary and Foreign Keys in iSQL Junior.*

Enter Statements:
```
show tables;
```
Execute Clear

**SELECT**
**TABLE_NAME**
**FROM**
**ALL_TABLES**
**WHERE OWNER =**
**(SELECT USER**
**FROM DUAL)**

| TABLE_NAME |
|---|
| RATING5520 |
| PLAYER5520 |
| MOVIE5520 |
| JOB5520 |
| DRIVER5520 |
| COLOURTYPE5520 |
| CASTING5520 |
| CAR5520 |
| CAMPAIGN5520 |
| ASSIGNMENT5520 |
| ACTOR5520 |

OK. 11 rows selected.

***Figure 41: Verification of Final Table List Including ASSIGNMENT5520.***

Enter Statements:
```
INSERT INTO ASSIGNMENT5520 (MobileNo, JobID, DoDate, Fee) VALUES
('0901234567', '47', DATE '2018-04-15', 10.00)
```
Execute Clear

OK. 1 rows

Enter Statements:

```
INSERT INTO ASSIGNMENT5520 (MobileNo, JobID, DoDate, Fee) VALUES
('0912345678', '52', DATE '2018-04-16', 15.00);
```

Execute Clear

OK. 1 rows

*Figure 42: Executing Insert Statement for ASSIGNMENT5520 in iSQL Junior.*

Enter Statements:

```
SELECT * FROM ASSIGNMENT5520
```

Execute Clear

**SELECT * FROM ASSIGNMENT5520**

| MOBILENO | JOBID | DODATE | FEE |
|---|---|---|---|
| 0901234567 | 47 | 15-APR-18 | 10 |
| 0912345678 | 52 | 16-APR-18 | 15 |

OK. 2 rows selected.

*Figure 43: Querying All Records from ASSIGNMENT5520 Table.*

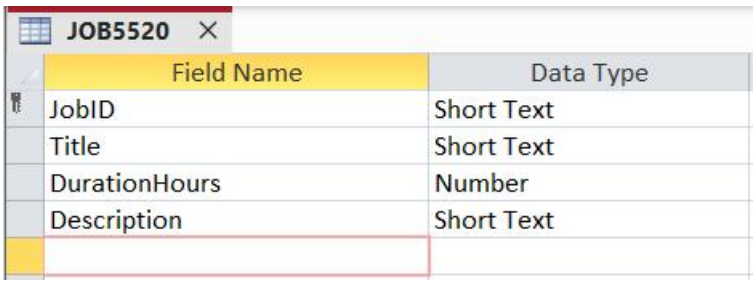



*Figure 44: JOB5520 Table Design View in Microsoft Access.*

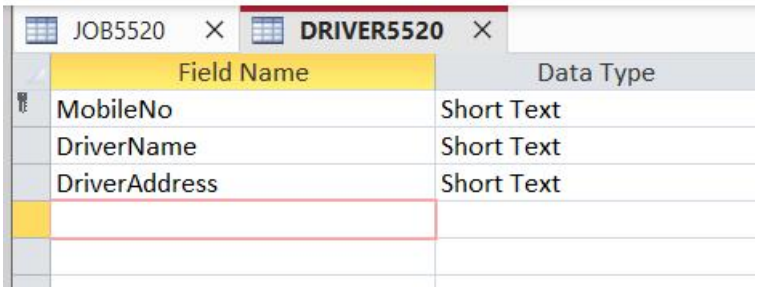



*Figure 45: DRIVER5520 Table Design View in Microsoft Access.*

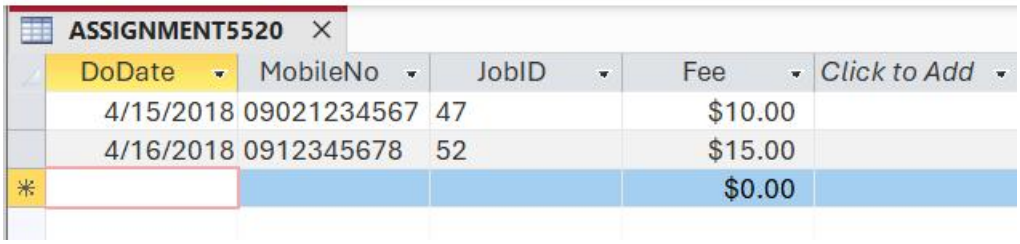



*Figure 46: ASSIGNMENT5520 Table Design View in Microsoft Access.*

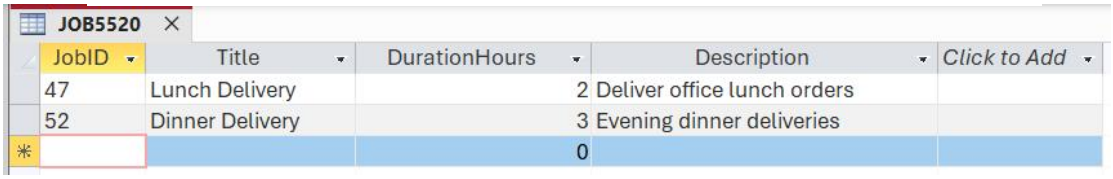



*Figure 47: JOB5520 Table Data Sheet View in Microsoft Access.*

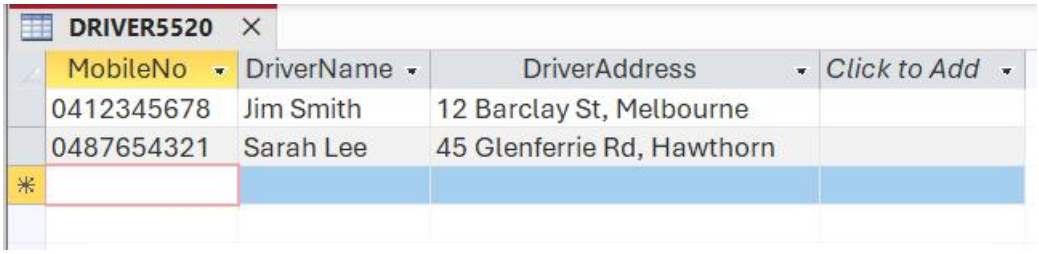



*Figure 48: DRIVER5520 Table Data Sheet View in Microsoft Access.*

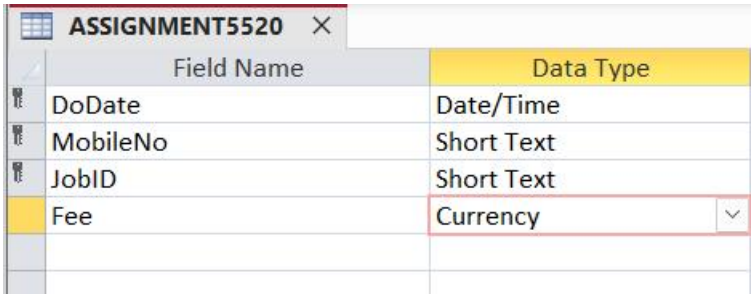



*Figure 49: ASSIGNMENT5520 Table Data Sheet View in Microsoft Access.*

ASSIGNMENT5520 - Notepad

File Edit View

```
CREATE TABLE ASSIGNMENT5520 (
  MobileNo    VARCHAR(50),
  JobID       VARCHAR(10),
  DoDate      DATE,
  Fee         NUMBER(6, 2),
  PRIMARY KEY (MobileNo, JobID, DoDate),
  FOREIGN KEY (MobileNo) REFERENCES DRIVER5520(MobileNo),
  FOREIGN KEY (JobID) REFERENCES JOB5520(JobID)
);
```

***Figure 50: SQL Script File for ASSIGNMENT5520 Table Creation.***

## K. Task Pass4k

***Scenario***: Hawthorn Comedy Inc. is planning to hold a festival made up of various comedy shows. Every show will have an ID, a title and a description. Each show is delivered by a comedian. Comedians have a name, an address, contact number, and email. One comedian may deliver more than one show at the festival. Members of the public will come and visit the festival as attendees. As each attendee enters the festival, their name, address and phone number will be recorded. Each attendee will be given a unique barcode id. During their visit, attendees will rate the show they see with a score of 1 to 10. This will be recorded by staff at Hawthorn Comedy Inc. Attendees will be able to vote for more than one show.

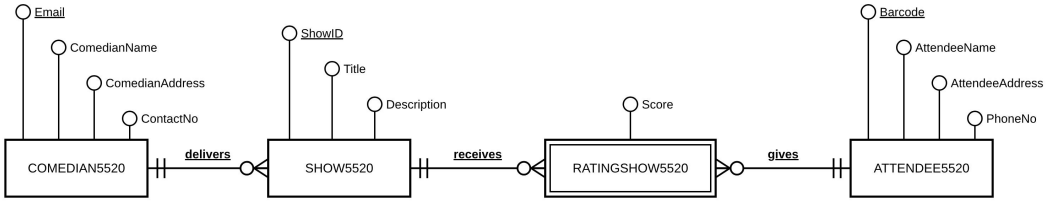



***Figure 51: Entity Relationship Diagram (ERD) for Comedian, Show, Rating, and Attendee Entities.***

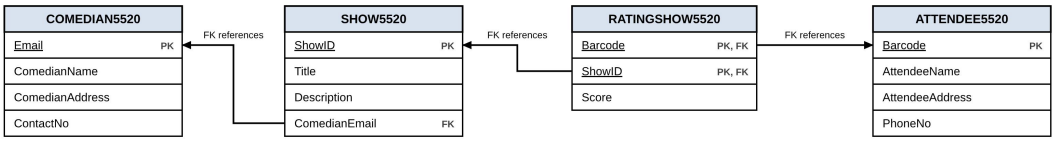



***Figure 52: Entity Relationship Model (ERM) for Comedian, Show, Rating, and Attendee Entities.***

COMEDIAN5520 (Email, ComedianName, ComedianAddress, ContactNo)
Primary Key (Email)

SHOW5520 (ShowID, Title, Description, ComedianEmail)
Primary Key (ShowID)
Foreign Key (ComedianEmail) references COMEDIAN5520

ATTENDEE5520 (Barcode, AttendeeName, AttendeeAddress, PhoneNo)
Primary Key (Barcode)

RATINGSHOW5520 (Barcode, ShowID, Score)
Primary Key (Barcode, ShowID)
Foreign Key (Barcode) references ATTENDEE5520
Foreign Key (ShowID) references SHOW5520

***Figure 53: Relational Schema for Comedian, Show, Rating, and Attendee Entities.***

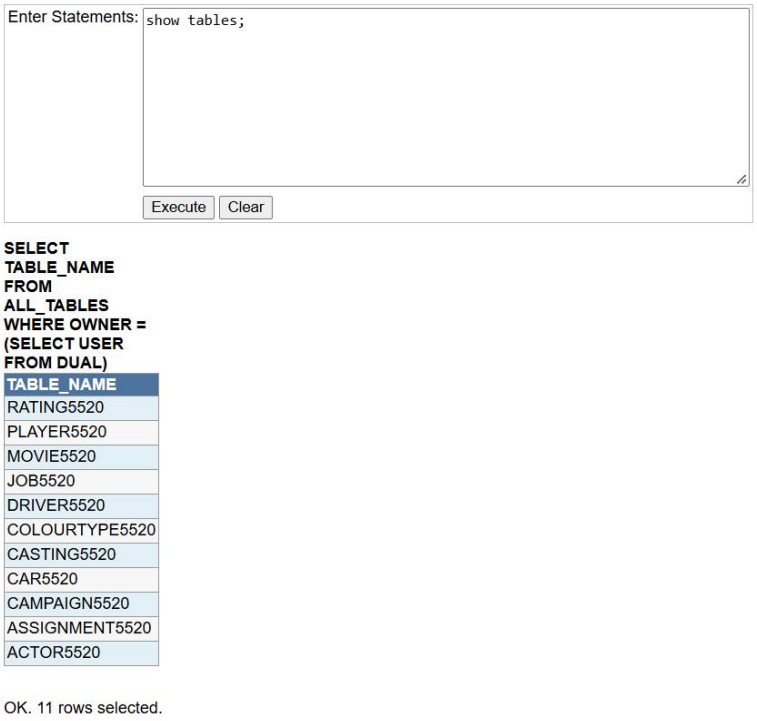
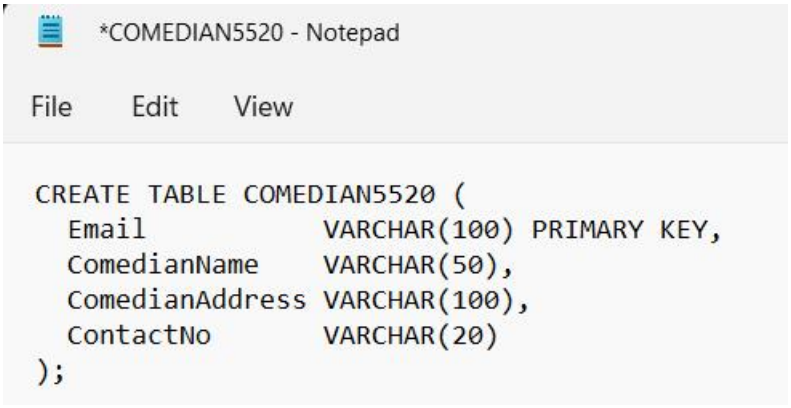


*Figure 54:*




***Verification of Existing Database Tables Before Creating the Pass 4k Tables in iSQL Junior.***

***Figure 55: SQL Script File Content for COMEDIAN5520 Schema.***

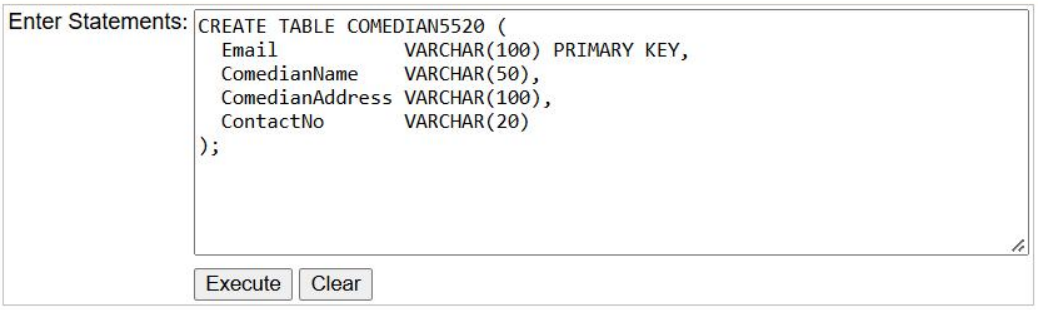



***Figure 56: Creating COMEDIAN5520 Table in iSQL Junior.***

SHOW5520 - Notepad

File Edit View

```
CREATE TABLE SHOW5520 (
  ShowID        VARCHAR(10) PRIMARY KEY,
  Title         VARCHAR(50),
  Description   VARCHAR(100),
  ComedianEmail VARCHAR(100),
  FOREIGN KEY (ComedianEmail) REFERENCES COMEDIAN5520(Email)
);
```

*Figure 57: SQL Script File Content for SHOW5520 Schema.*

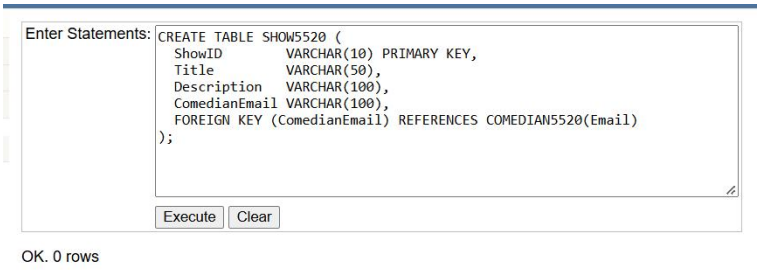



*Figure 58: Creating SHOW5520 Table in iSQL Junior.*

ATTENDEE5520 - Notepad

File Edit View

```
CREATE TABLE ATTENDEE5520 (
  Barcode         VARCHAR(20) PRIMARY KEY,
  AttendeeName    VARCHAR(50),
  AttendeeAddress VARCHAR(100),
  PhoneNo         VARCHAR(20)
);
```

*Figure 59: SQL Script File Content for ATTENDEE5520 Schema.*

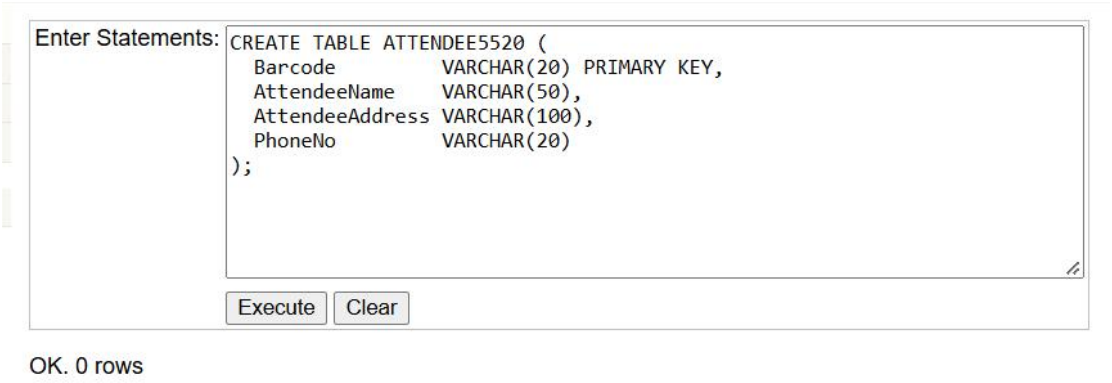



*Figure 60: Creating ATTENDEE5520 Table in iSQL Junior.*

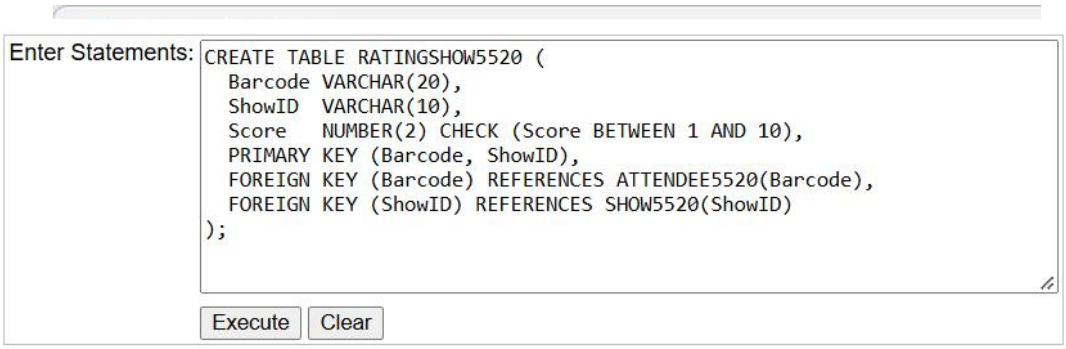
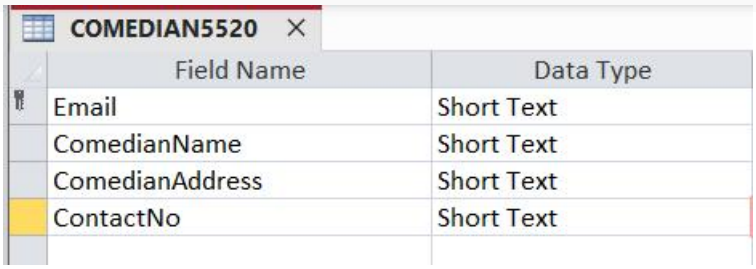



*Figure 61: SQL Script for File Content*




*RATINGSHOW5520 Schema.*

*Figure 62: Creating RATINGSHOW5520 Table in iSQL Junior.*

*Figure 63: COMEDIAN5520 Table Design View in Microsoft Access.*

*Figure 64: COMEDIAN5520 Table Datasheet View in Microsoft Access.*

ATTENDEE5520

| Field Name | Data Type |
|---|---|
| Barcode | Short Text |
| AttendeeName | Short Text |
| AttendeeAddress | Short Text |
| PhoneNo | Short Text |
| | |

*Figure 65: SHOW5520 Table Design View in Microsoft Access.*

SHOW5520

| ShowID | Title | Description | ComedianEmail | Click to Add |
|---|---|---|---|---|
| S01 | Laugn Out Loud | An evening of stand-up | anna.lee@example.com | |
| S02 | Late Night Giggles | Late night improv comedy | tom.reed@example.com | |
| | | | | |

*Figure 66: SHOW5520 Table Datasheet View in Microsoft Access.*

COMEDIAN5520

| Email | ComedianName | ComedianAddress | ContactNo | Click to Add |
|---|---|---|---|---|
| anna.lee@example.com | Anna Lee | 10 Burwood Rd, Hawthorn | 041111111 | |
| tom.reed@example.com | Tom Reed | 22 Glenferrie Rd, Hawthorn | 042222222 | |
| | | | | |

*Figure 67: ATTENDEE5520 Table Design View in Microsoft Access.*

ATTENDEE5520

| Barcode | AttendeeName | AttendeeAddress | PhoneNo | Click to Add |
|---|---|---|---|---|
| B1001 | Minh Tran | 5 Camberwell Rd, Camberwell | 093333333 | |
| B1002 | Linh Pham | 8 Riverside Rd, Hawthorn | 094444444 | |
| | | | | |

*Figure 68: ATTENDEE5520 Table Datasheet View in Microsoft Access.*

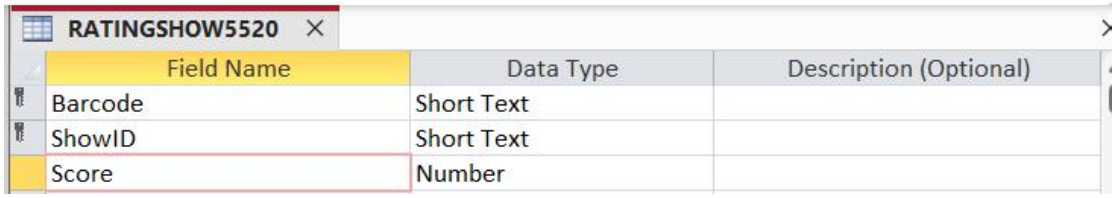



*Figure 69: RATINGSHOW5520 Table Design View in Microsoft Access.*

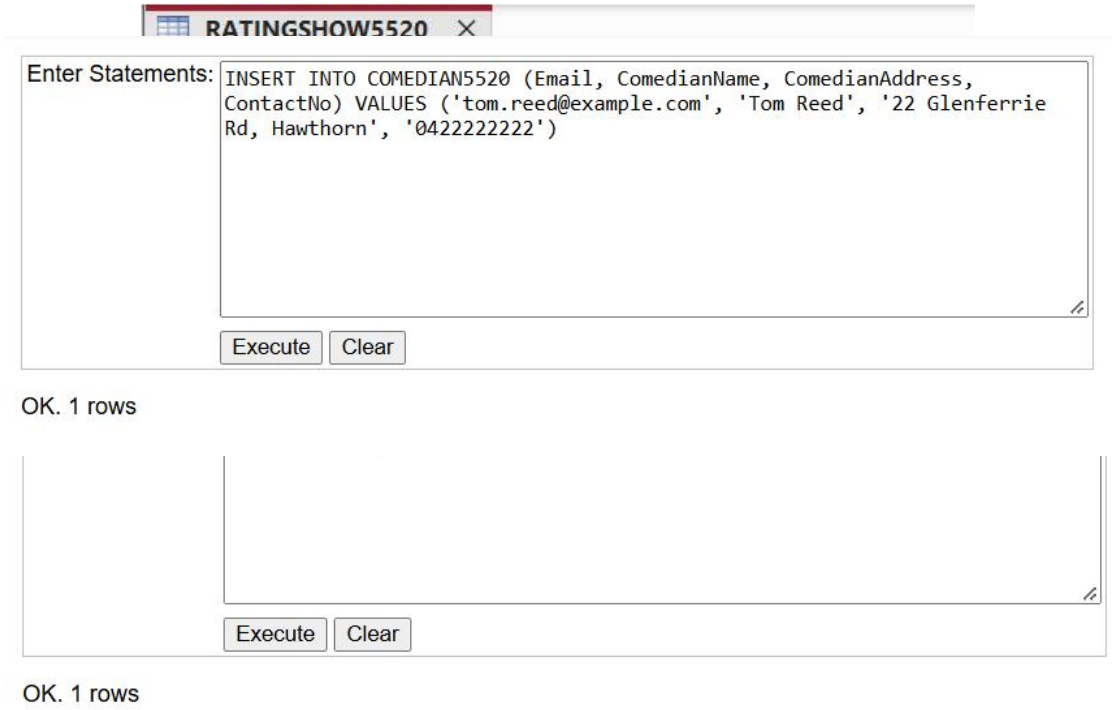



*Figure 70: RATINGSHOW5520 Table Datasheet View in Microsoft Access.*

*Figure 71: Executing Insert Statement for COMEDIAN5520 Table.*

Enter Statements:
```
SELECT * FROM COMEDIAN5520
```
Execute Clear

**SELECT * FROM COMEDIAN5520**

| EMAIL | COMEDIANNAME | COMEDIANADDRESS | CONTACTNO |
|---|---|---|---|
| anna.lee@example.com | Anna Lee | 10 Burwood Rd, Hawthorn | 0411111111 |
| tom.reed@example.com | Tom Reed | 22 Glenferrie Rd, Hawthorn | 0422222222 |

OK. 2 rows selected.

*Figure 72: Querying All Records from COMEDIAN5520 Table.*

Enter Statements:
```
INSERT INTO SHOW5520 (ShowID, Title, Description, ComedianEmail) VALUES
('S01', 'Laugh Out Loud', 'An evening of stand-up',
'anna.lee@example.com')
```
Execute Clear

OK. 1 rows

Enter Statements:
```
INSERT INTO SHOW5520 (ShowID, Title, Description, ComedianEmail) VALUES
('S02', 'Late Night Giggles', 'Late night improv comedy',
'tom.reed@example.com')
```
Execute Clear

OK. 1 rows

*Figure 73: Executing Insert Statement for SHOW5520 Table.*

*Figure 74: Querying All Records from SHOW5520 Table.*

Enter Statements:

```
INSERT INTO ATTENDEE5520 (Barcode, AttendeeName, AttendeeAddress,
PhoneNo) VALUES ('B1002', 'Linh Pham', '8 Riversdale Rd, Hawthorn',
'0944444444')
```

Execute Clear

OK. 1 rows

Enter Statements:

```
SELECT * FROM SHOW5520
```

Execute Clear

**SELECT * FROM SHOW5520**

| SHOWID | TITLE | DESCRIPTION | COMEDIANEMAIL |
|---|---|---|---|
| S01 | Laugh Out Loud | An evening of stand-up | anna.lee@example.com |
| S02 | Late Night Giggles | Late night improv comedy | tom.reed@example.com |

OK. 2 rows selected.

*Figure 75: Executing Insert Statement for ATTENDEE5520 Table.*

**SELECT * FROM ATTENDEE5520**

| BARCODE | ATTENDEENAME | ATTENDEEADDRESS | PHONENO |
|---|---|---|---|
| B1002 | Linh Pham | 8 Riversdale Rd, Hawthorn | 0944444444 |
| B1001 | Minh Tran | 5 Camberwell Rd, Camberwell | 0933333333 |

OK. 2 rows selected.

*Figure 76: Querying All Records from ATTENDEE5520 Table.*

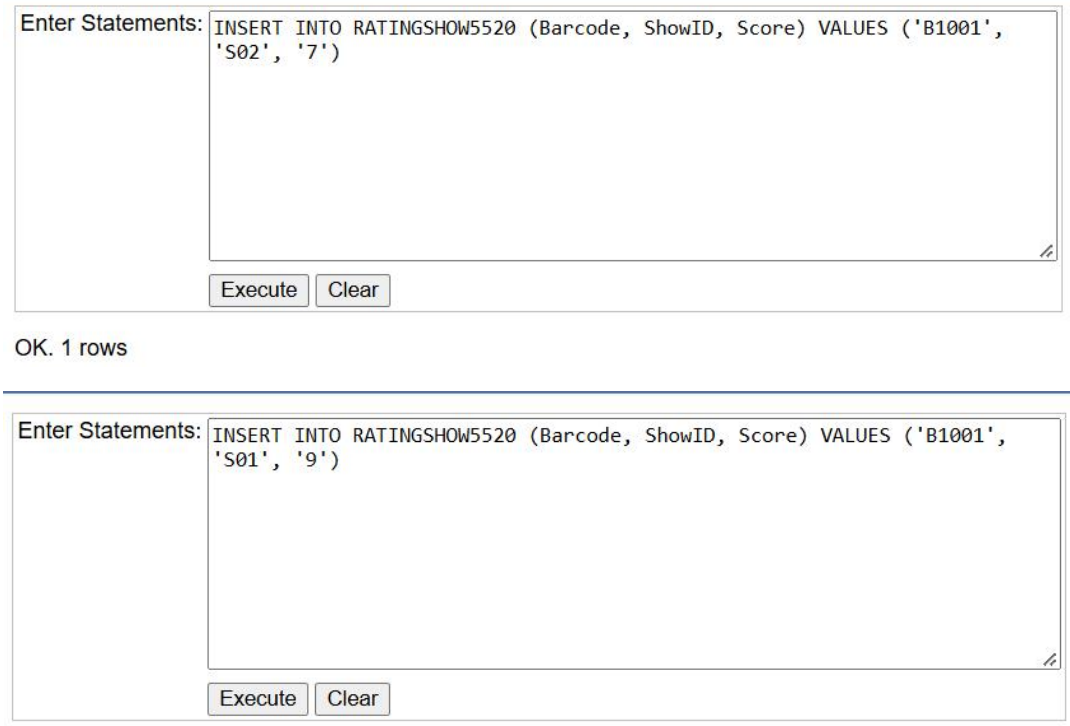



*Figure 77: Executing Insert Statement for RATINGSHOW5520 Table.*

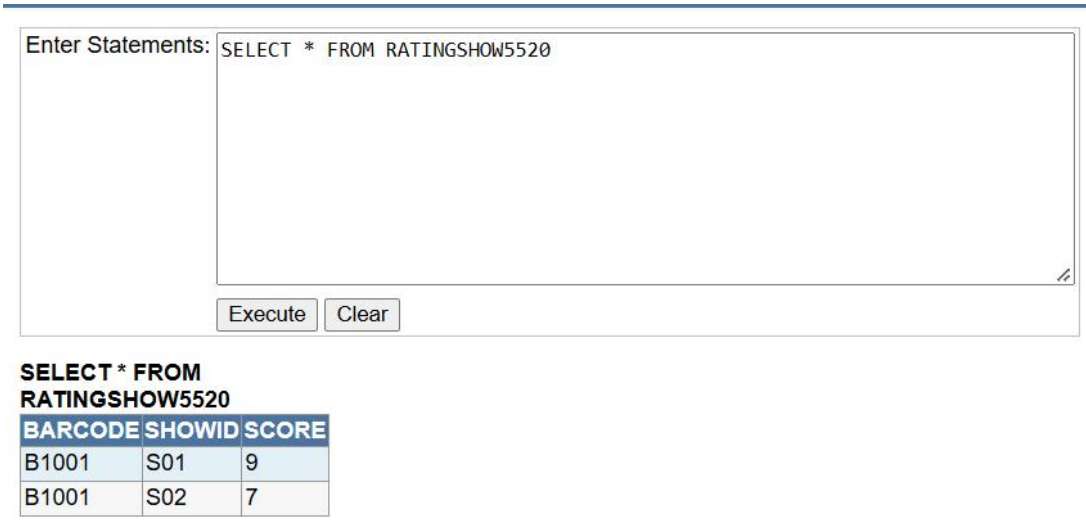



*Figure 78: Querying All Records from RATINGSHOW5520 Table.*

*Figure 79: Script File Containing DDL and Insert Statements for COMEDIAN5520.*

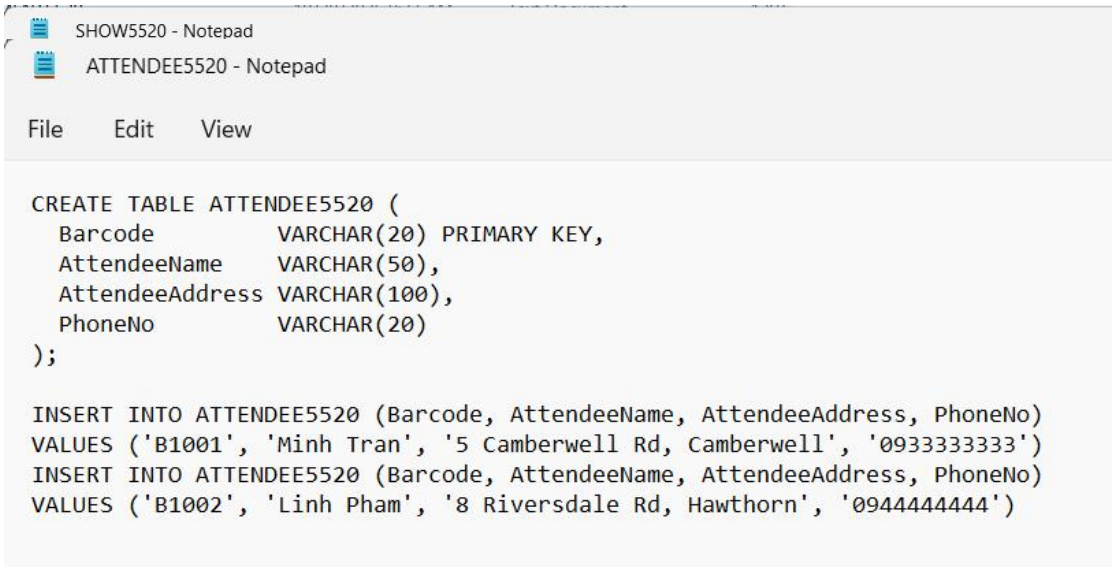



*Figure 80: Script File Containing DDL and Insert Statements for SHOW5520.*




*Figure 81: Script File Containing DDL and Insert Statements for ATTENDEE5520.*

RATING5520 - Notepad

File Edit View

```
CREATE TABLE RATINGSHOW5520 (
  Barcode VARCHAR(20),
  ShowID  VARCHAR(10),
  Score   NUMBER(2) CHECK (Score BETWEEN 1 AND 10),
  PRIMARY KEY (Barcode, ShowID),
  FOREIGN KEY (Barcode) REFERENCES ATTENDEE5520(Barcode),
  FOREIGN KEY (ShowID) REFERENCES SHOW5520(ShowID)
);

INSERT INTO RATINGSHOW5520 (Barcode, ShowID, Score) VALUES ('B1001', 'S01', '9')
INSERT INTO RATINGSHOW5520 (Barcode, ShowID, Score) VALUES ('B1001', 'S02', '7')
```

***Figure 82: Script File Containing DDL and Insert Statements for RATINGSHOW5520.***

## Conclusion

From my perspective, Task 4 Pass strengthened my understanding of how an entity-relationship diagram is converted into a relational schema, particularly how one-to-many relationships create foreign keys and how many-to-many relationships are resolved through a weak entity with a composite primary key. Moreover, creating the tables in iSQL Junior and deliberately testing the primary key, foreign key and parent-child constraints showed me how a database enforces referential integrity in practice. Consequently, I became more confident in writing SQL statements, including joins, aggregate functions and the IN operator in Access, to retrieve meaningful information from related tables.